Bloom Filters in Duplicate Detection of URL's using Web Crawlers

Data Structures and Advanced Algorithms - Assignment 2

Gonçalo Gonçalves da Costa Pinto
Manuel Mo
Pedro Rafael Correia Borges

Master in Informatics and Computing Engineering

Professor: Jácome Miguel Costa da Cunha

3 of June of 2026

Contents

1 Introduction

In the present days, the internet is an essential infrastructure for communication, commerce, and information retrieval. Web crawlers are a fundamental component of this ecosystem, responsible for systematically browsing the web to index content for search engines and other large-scale data collection systems. At the scale of billions of pages, one of the most already-visited URLs are not refetched unnecessarily, which would waste bandwidth, CPU time, and storage.

The naive approach to this problem is to maintain a hash set of all visited URLs. However, storing full URL strings in memory becomes prohibitively expensive at scale. For 500,000 URLs, a standard `std::unordered_set` consumes over 30 MB of memory. At Google's crawl scale of approximately 5 billion pages, this approach would required hundreds of gigabytes solely for the visited URL set.

This report presents a probabilistic alternative based on Bloom filters and their variants. It explores three data structures: the Standard Bloom filter, the Counting Bloom filter, and the Scalable Bloom filter, and evaluate their applicability to the web crawling problem. The implementation, written in C++, achieves under 1 MB of memory for 500,000 URLs while maintaining a false positive rate below 0.1%, with O(k) lookup time independent of the number of stored elements.

2 Background & Related Work

The Bloom filter was originally proposed by Burton Howard Bloom in 1970 as a space-efficient probabilistic data structure for set membership queries [1]. It uses a bit array of m bits and k independent hash functions to represent a set of elements. Adding an element sets k bits in the array, and membership is tested by checking whether all k corresponding bits are set. This design guarantees no false negatives: if an element was inserted, it will always be recognized as a member. However, false positives are possible, as multiple elements may collectively set the same combination of bits.

The optimal number of hash functions is given by $k = \frac{m}{n} \ln 2$, and the resulting false positive probability is approximately $p = \left(1 - e^{-kn/m}\right)^k$, where n is the number of inserted elements. In practice, allocating roughly 10 bits per expected element yields a false positive rate well below 1%.

The Standard Bloom filter does not support deletions, since clearing a bit might remove information contributed by other elements. The Counting Bloom filter, introduced by Fan et al. [2], addresses this limitation by replacing each bit with a small counter, typically 4 bits. Incrementing the counter on insertion and decrementing on deletion enables safe removal of elements, at the cost of approximately four times (if 4 bits) the memory of the standard variant.

For scenarios where the total number of elements is not known in advance, Almeida et al. [3] proposed the Scalable Bloom Filter (SBF). Rather than pre-allocating a fixed size structure, the SBF maintains a chain of Standard Bloom filters. When the current filter reaches its capacity, a new, larger filter is appended with a tighter false positive target. Each successive filter grows by a scale factor s (typically 2) and tightens its false positive rate by a ratio r (typically 0.85), guaranteeing a bounded global false positive rate of $p_0 \cdot \frac{1}{1-r}$ regardless of growth.

Bloom filters have seen wide adoption in systems requiring fast, memory-efficient membership testing. Google's BigTable uses them to avoid unnecessary disk lookups [4], Apache Cassandra employs them to reduce I/O in its storage engine, and Google Chrome originally used a Bloom filter to check URLs against a list of malicious sites before querying a remote server. In the web crawling literature, Bloom filters have been a standard tool for visited-URL tracking since at least the early work of Heydon and Najork [5].

3 Data Structures

The core contribution of this work is the implementation and comparative evaluation of three Bloom filter variants for the specific problem of duplicate URL detection in web crawling. All three structures share the same fundamental operations: insertion of an element (URL) and membership query. However, each variant offers distinct trade-offs between memory consumption, operational flexibility (support for deletions), and adaptability to unknown dataset sizes. The following subsections detail the design choices, implementation strategies, and theoretical guarantees for each variant.

3.1 Standard Bloom Filter

A Bloom filter consists of a bit array of m bits, initialized to zero, and k hash functions mapping elements to positions in the array. To add an element x, the bits at positions $h_1(x), h_2(x), \ldots, h_k(x)$ are set to 1. To query membership, all k positions are checked. The element is considered present if and only if all bits are set.

In the implementation of this project, double hashing is used to derive k independent hash functions from a single base hash, computed with the Splitmix64 algorithm [6]. This avoids the overhead of k independent hash computations while maintaining sufficient independence in practice. Given a target false positive rate p and expected element count n, the optimal bit array size is:

$$m = -\frac{n \ln p}{(\ln 2)^2}$$

and the optimal number of hash functions is $k = \frac{m}{n} \ln 2$.

3.2 Counting Bloom Filter

The Counting Bloom filter extends the standard variant by replacing each bit with a 4-bit counter, supporting values from 0 to 15. Addition increments the k counters corresponding to an element (removal decrements them). A position is logically set when its counter is nonzero. To minimize memory overhead, two 4-bit counters are packed into a single byte using bitwise operations.

Counter overflow is handled by saturation: a counter that reaches 15 does not overflow, preventing incorrect deletions at the cost of potentially refusing a future decrement. This situation is extremely rare in practice under normal load conditions. The memory cost is approximately four times that of the Standard Bloom filter: for 500,000 URLs, this amounts to approximately 2,441 KB, compared to 610 KB for the standard variant: still roughly 12 times smaller than an `unordered_set`.

The primary use case for the Counting Bloom filter in a web crawler is handling URL expiry: pages that return HTTP 404, domains that become unreachable, or URLs whose cached content has expired can be removed from the filter and potentially requeued for crawling. Without deletion support, a Standard Bloom filter would require periodic reconstruction to avoid accumulating stale entries: an expensive operation at scale.

3.3 Scalable Bloom Filter

The Scalable Bloom Filter addresses the case where the number of URLs to be crawled is not known in advance. It maintains an ordered list of Standard Bloom filters, each with a progressively larger bit array and a tighter false positive target. New elements are always inserted into the most recent (active) filter. When the active filter reaches its designed capacity, determined by its load factor, a new filter is created with size $m_i = m_0 \cdot s^i$ and false positive target $p_i = p_0 \cdot r^i$.

Membership queries must check all filters in the chain, returning true if any filter reports membership. The global false positive rate is bounded by the geometric series:

$$P_{\text{global}} \leq p_0 \cdot \sum_{i=0}^{\infty} r^i = \frac{p_0}{1 - r}$$

4 Implementation

The implementation is written in C++ as a set of header-only modules with no external dependencies, organized across four files [7].

4.1 bloom_filter.hpp

This header defines the three filter classes: `BloomFilter`, `CountingBloomFilter`, and `scalableBloomFilter`, along with the helper functions `optimalBloomParams()` and `theoreticalFpRate()`.

`optimalBloomParams(n, p)` takes the expected number of insertions n and a desired false positive rate p and returns the optimal bit count m and hash count k according to the standard formulas:

$$m = \left\lceil -\frac{n \ln p}{(\ln 2)^2} \right\rceil, \qquad k = \left\lfloor \frac{m}{n} \ln 2 \right\rceil.$$

All three filters share the same hashing strategy. Rather than computing k independent hash functions, double hashing derives all k positions from a single base hash. Given the raw hash h_1 of a string, a second value h_2 is produced with the Splitmix64 mixer [6], and the i-th hash position is computed as $h_1 + i \cdot h_2 + i^2 \pmod m$. This provides sufficient independence in practice while avoiding the cost of k separate hash evaluations.

BloomFilter: The standard filter stores a packed bit array of $\lceil m/8 \rceil$ bytes. `add(x)` sets k bits, `contains(x)` returns true iff all k bits are set.

CountingBloomFilter: Each position holds a 4-bit counter rather than a single bit, supporting values from 0 to 15. Two counters are packed into each byte using bitwise operations, giving a storage overhead of exactly $\times 4$ compared to the standard filter. `add(x)` increments the k counters. `remove(x)` decrements them, first verifying membership to avoid removing an element that was never inserted. Counter overflow saturates at 15 rather than wrapping, preventing spurious decrements at the cost of refusing a future removal for that position: an event that is negligibly rare under normal load.

ScalableBloomFilter: The scalable filter maintains a `std::vector` of `Layer` structs, each wrapping a `BloomFilter` with its own expected insertion count and false positive target. A new layer is appended whenever the active layer reaches its designed capacity. The per-layer false positive targets are set so that their sum converges: with tightening ratio r and initial target p_0, the target for layer i is $p_0 \cdot (1 - r) \cdot r^i$, giving a geometric series that sums to p_0. `contains(x)` queries all layers and returns true if any reports membership.

4.2 url_utils.hpp

This header provides two functions: `isBadUrl()` and `normalizeUrl()`.

`isBadUrl()` rejects URLs that are empty, exceed 2048 characters, contain whitespace, use a scheme other than `http` or `https`, or have no host component.

`normalizeUrl()` converts a URL to a canonical form before it is hashed and inserted into any filter. Without normalization, logically identical URLs such as `HTTPS://Example.COM/` and `https://example.com` would be treated as distinct, inflating the filter with redundant entries. The normalization pipeline applies five transformations in order:

1. Lowercase the scheme and host components.
2. Strip default ports (`:80` for HTTP, `:443` for HTTPS).
3. Remove the fragment identifier (`#section`).
4. Normalize percent-encoding: decode unreserved characters and uppercase hex digits in remaining `%XX` sequences.
5. Remove a bare trailing slash from the path (`example.org/` → `example.org`).

The function returns `std::nullopt` if the URL is invalid.

4.3 web_crawler.hpp

The central data structure in this header is the class template `BasicWebCrawler<BloomType>`, which is parameterized on the filter type. Three type aliases are provided at the bottom of the header:

```
using WebCrawler         = BasicWebCrawler<BloomFilter>;
using CountingWebCrawler = BasicWebCrawler<CountingBloomFilter>;
using ScalableWebCrawler = BasicWebCrawler<scalableBloomFilter>;
```

This design allows the same crawler logic to be evaluated with any of the three filter variants without code duplication. The constructor uses `if constexpr` to dispatch on the filter type: fixed size filters (`BloomFilter` and `CountingBloomFilter`) are constructed from the optimal bit count and hash count derived via `optimalBloomParams`, while `scalableBloomFilter` is constructed directly from the expected page count and desired false positive rate.

The crawler performs a multi-threaded breadth-first traversal. Its behaviour is governed by a `CrawlerConfig` struct specifying the thread count, maximum crawl depth, maximum pages, politeness delay in milliseconds, a set of allowed domains, and a set of blocked file extensions. Optional callbacks `onPageFetched` and `onProgress` allow the caller to observe results in real time.

Worker threads share a URL queue protected by a mutex and condition variable. Each thread dequeues a `QueuedUrl`, checks whether it should be crawled, enforces a per-domain politeness delay, fetches the page over a raw TCP socket, extracts outbound links via regular expressions applied to the response body, normalizes them with `normalizeAndValidate()`, and enqueues those within scope and within the depth limit.

Duplicate detection uses a two-level strategy. The Bloom filter provides a fast probabilistic first gate: if `contains(url)` returns false, the URL is definitely new and proceeds. If it returns true, the result is verified against a secondary `std::unordered_set` of confirmed visited URLs held in `visitedSet_`. This hybrid approach preserves the memory advantage of the Bloom filter for the common case while ensuring that no URL is silently skipped due to a false positive.

Robots.txt compliance is handled by fetching and parsing the `/robots.txt` file for each new domain on first contact and caching the result in the `domains_` map. Every URL is checked against the disallow list for its domain before being fetched. If a `Crawl-delay` directive is present, the politeness delay for that domain is set to the larger of the configured minimum and the directive value.

4.4 crawler_demo.cpp

This file provides the entry point for live crawling experiments. It accepts positional and named arguments that select the filter variant and configure the crawl:

```
./crawler_demo [standard|counting|scalable] [options]
  --seconds N        Crawl duration in seconds   (default: 30)
  --threads N        Worker threads              (default: 4)
  --depth N          Maximum crawl depth         (default: 3)
  --pages N          Maximum pages               (default: 100)
  --delay MS         Per-domain politeness delay (default: 1000 ms)
  --allow-domain D   Restrict crawl to domain D  (repeatable)
  --seed URL         Add a seed URL              (repeatable)
```

The `runCrawlerDemo` function template is parameterized on the crawler type, allowing all three variants to share the same setup, callback, and reporting code. The `onPageFetched` callback prints each fetched URL with its HTTP status code, fetch time, and content type. The `onProgress` callback writes a live counter to standard output. On completion, `printCrawlerStats()` reports pages crawled, pages failed, duplicates skipped, out-of-scope URLs, robots-blocked URLs, total links found, elapsed time, and pages per second. `printBloomStats()` then reports the filter's memory usage, insertion count, and theoretical false positive rate, adapting its output to whether the filter is fixed size or scalable.

If no seeds are provided on the command line, the demo defaults to three `httpbin.org` endpoints, which serve deterministic HTML responses suitable for testing link extraction and duplicate detection without requiring a large external crawl.

4.5 main.cpp

The benchmark runner evaluates all three filter variants against a hash set baseline on large synthetic or file-loaded datasets, independently of the live crawler. It generates up to synthetic URLs with a default 20% duplicate rate, or loads URLs from a plain-text file if one is provided.

False positives are counted by comparing each filter's response against a ground-truth `std::unordered_set`: a false positive occurs when the filter reports membership for a URL that the set confirms has not been

inserted. The deletion capability of the counting filter is demonstrated by removing every fifth unique URL after the insertion phase and verifying absence via a subsequent `contains()` call.

4.6 Build and usage

All headers are self-contained and require only a C++17-compliant compiler. To build all the binary files in order to run the code, simply execute the script in the terminal: `./build.sh`

To test the generated files, below are some commands that can be used:

```
# Run the default benchmark
./build/url_crawler_benchmark

# Run the benchmark with a specific input size (e.g., 5,000,000)
./build/url_crawler_benchmark 5000000

# Run the benchmark using a specific input file
./build/url_crawler_benchmark 0 <filename>

# Run the standard crawler with custom configurations
./build/url_crawler standard \
  --seconds 60 \
  --threads 6 \
  --depth 3 \
  --pages 2000 \
  --delay 200 \
  --allow-domain httpbin.org \
  --seed http://httpbin.org/links/50/0
```

5 Empirical Assessment

Despite the crawler functioning correctly in terms of duplicate detection and robots.txt compliance, practical limitations prevented the group from collecting large-scale empirical data from live crawling. First, evaluating the false positive rate in a live environment is conceptually problematic, as there is no definitive ground truth. Without maintaining a massive, memory-heavy list of every single visited URL, it is impossible to verify whether a flagged duplicate is a genuine match or a false positive. Second, even if this could be tracked, the combination of per-domain politeness delays and the inherent latency of network requests makes it executionally impractical to gather the hundreds of thousands of distinct URLs required for statistically significant measurements.

For these reasons, the false positive rate analysis presented in the preceding subsections is based entirely on synthetic datasets, which allowed a controlled, verifiable, and reproducible evaluation at scales up to 2,000,000 URLs (could be a lot more). The live crawler experiments served instead to validate the correctness of the system's integration, confirming that duplicate URLs were successfully filtered, that the Bloom filter and visited set remained consistent, and that the politeness and robots.txt mechanisms behaved exactly as intended.

Note: All graphs presented in this section have their corresponding data tables included in the appendices.

5.1 Standard Bloom filter vs. Scalable Bloom filter

A key question when deploying Bloom filters in a web crawler is how to handle scenarios where the total number of URLs is not known in advance. The Standard Bloom filter requires a fixed size m to be allocated at construction time, based on an estimate of n. If this estimate is too low, the false positive rate degrades rapidly as the filter fills beyond its design capacity. The Scalable Bloom filter, by contrast, grows dynamically by appending new layers, maintaining a bounded false positive rate regardless of the total number of inserted elements.

Figure 1 shows the false positive rate of the Standard Bloom filter for three different fixed bit array sizes: 500,000 bits, 1,000,000 bits, and 5,000,000 bits. Each curve plots the FP rate as a function of the number of inserted URLs, evaluated at 100,000, 300,000, 500,000, 1,000,000, and 2,000,000 insertions.

For the smallest filter (500,000 bits), the FP rate rises sharply even at modest insertion counts. When the number of inserted URLs (500,000) equals the number of bits, the FP rate approaches approximately 50%. This behaviour is expected from Bloom filter theory: when $m = n$, regardless of k, the filter becomes densely populated and false positives become highly likely. When using the medium filter (1,000,000 bits) the FP rate is also around 50% at 1,000,000 URLs.

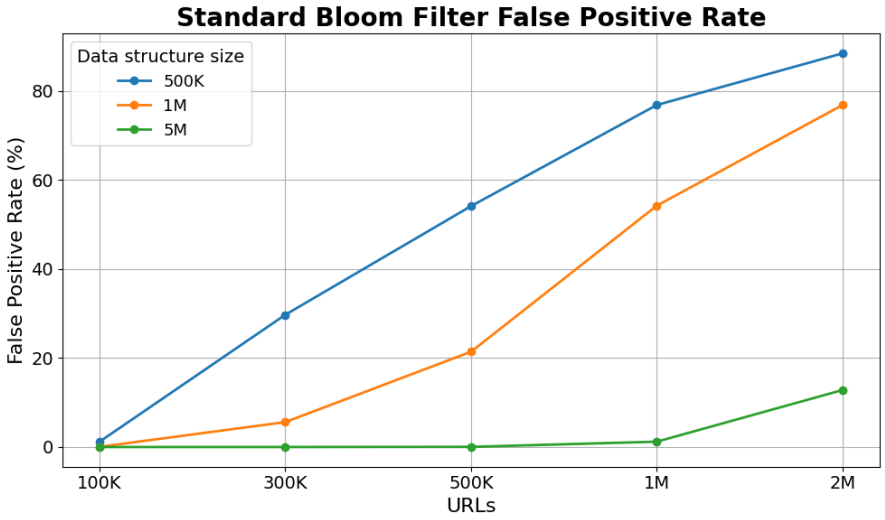

Figure 1: False positive rate of the Standard Bloom filter as a function of inserted URLs, for three different sizes (Table 8).

The graph below (Figure 2) presents the false positive rate of the Scalable Bloom filter under identical test conditions, with an initial target FP rate of 1%, scale factor $s = 2$, and tightening ratio $r = 0.90$. Unlike the standard variant, the scalable filter maintains a remarkably stable false positive rate as the number of URLs grows from 100,000 to 2,000,000. The FP rate starts at approximately 0.15% and decreases slightly to around 0.13% at 2 million URLs.

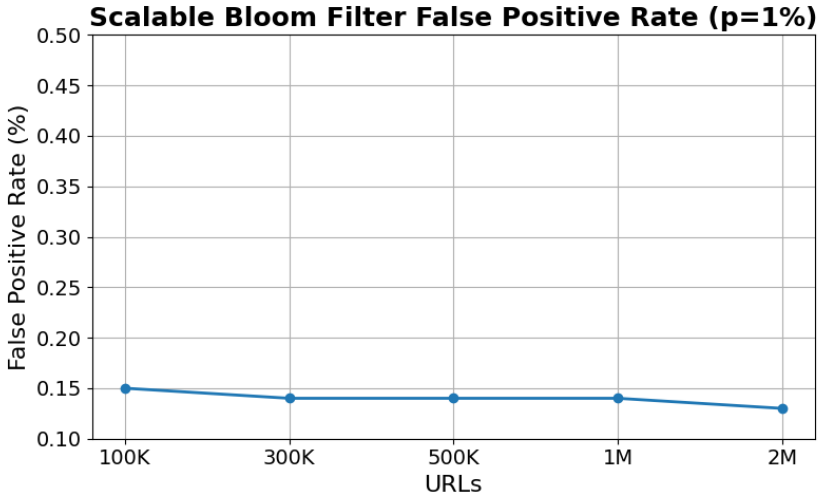

Figure 2: False positive rate of the Scalable Bloom filter as a function of inserted URLs, with initial target FP rate of 1%, scale factor $s = 2$, and tightening ratio $r = 0.90$ (Table 8).

This stable behaviour is a direct consequence of the Scalable filter's design. When the active layer reaches its capacity (determined by its configured fill ratio) a new layer is appended with double the size (scale factor $s = 2$) and a false positive target tightened by factor $r = 0.90$. The geometric series of per-layer FP rates ensures that the global false positive probability remains bounded.

The slight downward trend observed in Figure 2 is explained by the tightening mechanism: each new layer has a progressively stricter false positive target. As the dataset grows beyond one million URLs, the filter creates a second layer. Membership queries must now check both layers, but the probability of a false positive is dominated by the tighter newer layer. Consequently, the overall FP rate actually decreases slightly as more layers are added, contrasting sharply with the Standard filter where the FP rate monotonically increases with n.

5.2 Different Target FP rates of the Scalable Bloom filter

It is evident that the Scalable Bloom Filter outperforms the standard Bloom Filter and is generally more suitable for this context of problem. However, careful consideration must be given to the choice of the target false positive rate. As shown in Figure 3, memory usage is directly influenced by the target FP rate: lower target FP rates require more memory. For instance, when a target FP rate of 25% is used, the resulting false positive rate is the highest among all configurations (approximately 3.5%), while memory consumption is the lowest. Conversely, a target FP rate of 0.1% results in the lowest false positive rate but the highest memory usage. Therefore, the selection of the target FP rate should be based on the application's requirements and available memory resources, aiming to achieve an appropriate **balance between false positive performance, memory consumption, and latency**.

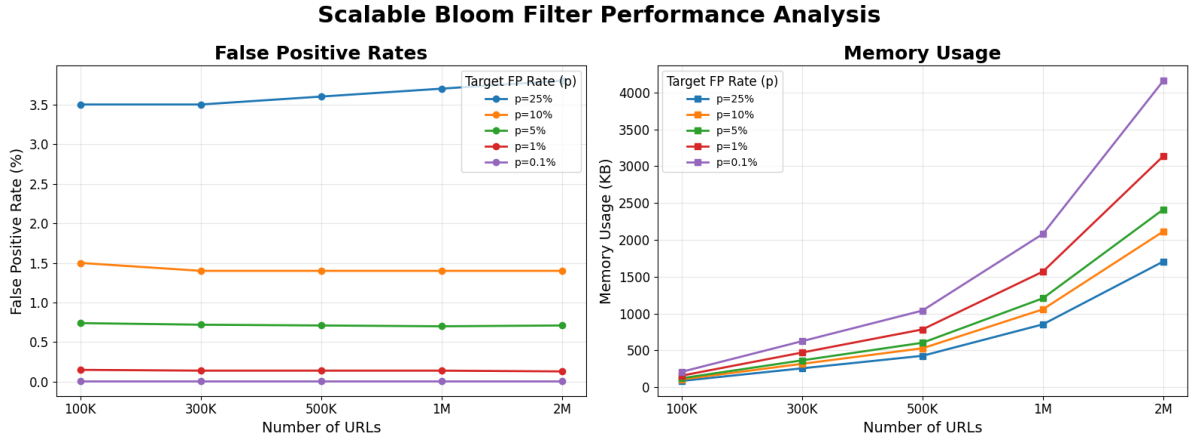

Figure 3: Evolution of false positive rates and memory usage in a Scalable Bloom Filter for different target FP rates (Table 8 and Table 8).

5.3 Memory Footprint Comparison

Comparing the Bloom filter variants with the hash set, Figure 4 shows that the hash set achieves perfect duplicate detection, with no false positives or false negatives, while also providing very competitive query performance. However, this accuracy comes at a significantly higher memory cost. The hash set requires approximately 29 MB of memory (as stated before), whereas both the Standard Bloom Filter and the Scalable Bloom Filter use less than 1 MB.

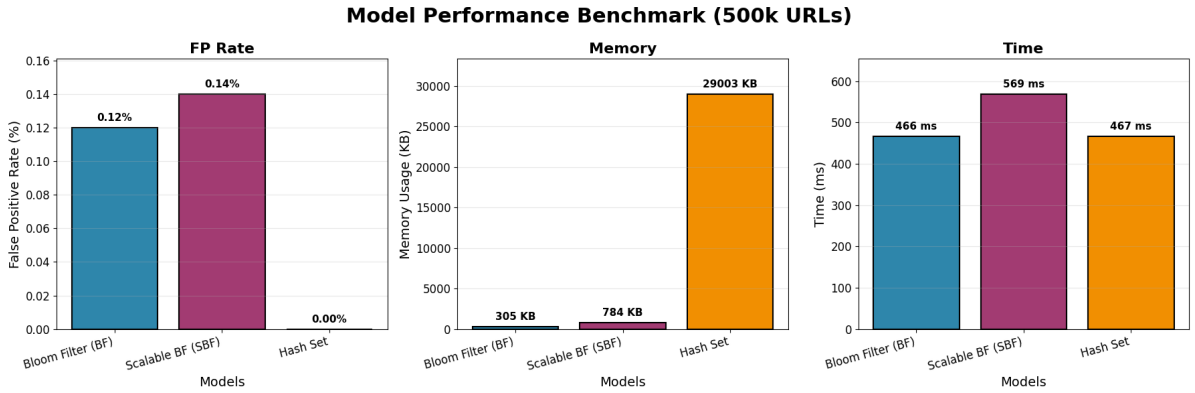

Figure 4: Performance comparison between the Standard Bloom Filter, Scalable Bloom Filter, and Hash Set (Table 8).

The choice between these data structures therefore depends on the application's requirements and resource constraints. When memory is abundant and exact duplicate detection is required, a hash set is the preferred solution. In contrast, when memory efficiency is a primary concern and a small false positive rate is acceptable, Bloom-filter-based approaches become more attractive. Among them, the Scalable Bloom Filter offers the best trade-off, as it maintains a bounded false positive rate while adapting dynamically to datasets whose final size is unknown.

In the context of large-scale web crawling, where billions of pages may need to be processed, memory efficiency becomes a critical factor. Under these conditions, the Scalable Bloom Filter emerges as the most suitable candidate, providing substantial memory savings while maintaining a low and predictable false positive rate.

5.4 Counting Bloom filter stats

The Counting Bloom Filter has not been included in the previous comparisons because its false positive behaviour is essentially identical to that of the Standard Bloom Filter. Figure 5 compares the false positive rates and memory usage of both data structures. As expected, the two filters exhibit nearly identical false positive rates, since they rely on the same underlying probabilistic membership mechanism.

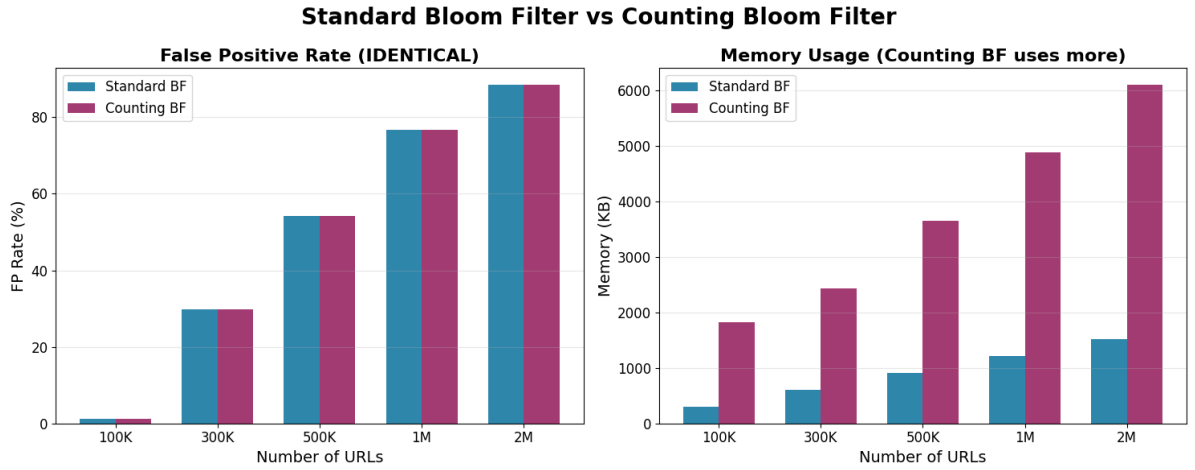

Figure 5: FP rate and memory usage evolutions between standard and counting bloom filter.

However, the Counting Bloom Filter requires substantially more memory. In the evaluated configuration, its memory consumption is approximately four times higher that of the Standard Bloom Filter, as each position stores a counter rather than a single bit. Consequently, the additional memory overhead does not provide any benefit when only membership queries are required.

The primary advantage of the Counting Bloom Filter is its support for element deletion. This feature is particularly useful in applications involving dynamic or temporal datasets, **where elements may expire or need to be removed over time**. In the context of web crawling, where URLs are typically inserted and retained throughout the crawling process, this capability provides limited practical benefit. Therefore, the Scalable Bloom Filter remains the most suitable alternative among the evaluated Bloom filter variants.

5.5 Live crawling results

Although the live crawling experiments do not provide statistically meaningful data for evaluating false positive rates, they are still valuable for validating the crawler's behaviour under realistic operating conditions. The following experiment was conducted using the configuration shown below.

```
./build/url_crawler standard \
  --seconds 60 \
  --threads 6 \
  --depth 3 \
  --pages 2000 \
  --delay 200 \
  --allow-domain httpbin.org \
  --seed http://httpbin.org/links/50/0
```

As shown by the results below, the crawler processed 53 unique pages in 60 seconds while skipping approximately 9.8 million duplicate URLs. This outcome reflects the highly interconnected structure of the test dataset, where the same URLs are repeatedly discovered through different navigation paths.

While these results demonstrate that duplicate detection is functioning correctly, they are not suitable for quantitative evaluation of Bloom filter performance. The limited number of unique pages, combined with network latency and politeness delays, prevents the collection of datasets large enough to produce statistically significant false positive measurements. Consequently, the controlled synthetic experiments presented in the previous sections provide a more reliable basis for comparing the evaluated data structures.

```
=== Standard Bloom ===
Starting crawler with 6 threads...
[Fetched] http://httpbin.org/links/50/0 (status: 200, time: 300ms, type: text/html;
charset=utf-8)
[Progress] Crawling: 1 pages, 0 failed[Fetched] http://httpbin.org/links/50/1 (status:
200, time: 294ms, type: text/html; charset=utf-8)
[Progress] Crawling: 2 pages, 0 failed[Fetched] http://httpbin.org/links/50/10 (status:
200, time: 552ms, type: text/html; charset=utf-8)
(...)
[Progress] Crawling: 47 pages, 0 failed[Fetched] http://httpbin.org/links/50/7 (status:
200, time: 303ms, type: text/html; charset=utf-8)
(...)
=== Crawler Statistics ===
Pages crawled      : 53
Pages failed       : 0
Duplicates skipped : 9814533
Out of scope       : 0
Robots blocked     : 0
Total links found  : 2597
Duration           : 60 seconds
Pages per second   : 0.88

=== Bloom Filter Stats ===
Bit count    : 19171
Hash count   : 7
Insertions   : 53
Memory (KB)  : 2
```

```
Theoretical FP: 0.000000
```

6 Critical Analysis

The empirical results presented in the previous sections highlight that no single data structure is optimal for every crawling scenario. Instead, the most appropriate choice depends on several factors, including memory constraints, whether the final number of URLs is known in advance, and whether support for URL deletion is required.

Based on these observations, Figure 6 summarizes the decision process for selecting the most suitable data structure. When memory usage is not a concern, a hash set provides exact duplicate detection with no false positives. However, when memory efficiency becomes important, Bloom-filter-based solutions offer significant advantages.

If the expected number of URLs is known and relatively stable, a Standard Bloom Filter is generally sufficient. In cases where URL deletion is required, a Counting Bloom Filter becomes preferable despite its higher memory overhead. Conversely, when the total number of URLs is unknown or may grow significantly over time, a Scalable Bloom Filter is a more appropriate choice, as it maintains a bounded false positive rate while dynamically expanding its capacity. If both scalability and deletion support are required, a Scalable Counting Bloom Filter provides the necessary functionality at the cost of additional memory consumption.

The flowchart therefore serves as a practical guideline for selecting a duplicate-detection data structure according to the requirements and constraints of a given web crawling application.

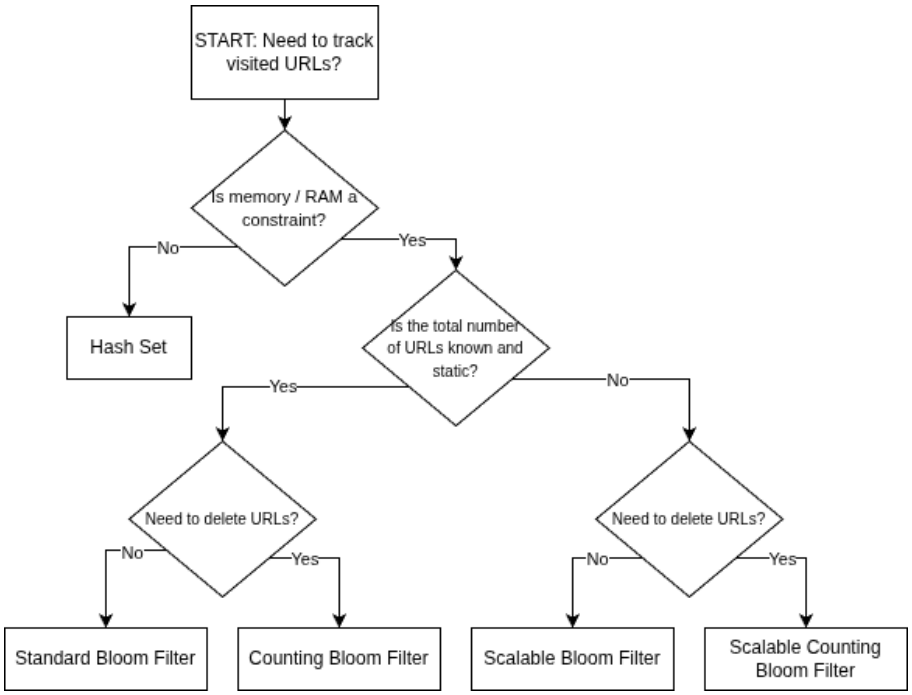

Figure 6: Decision flowchart for selecting an appropriate URL duplicate-detection data structure.

Although the flowchart provides a useful guideline, it should not be interpreted as a strict decision rule. The suitability of a particular data structure ultimately depends on the specific characteristics of the crawling workload and the system's operational constraints. For example, while a hash set offers perfect duplicate detection, its memory requirements may become prohibitive when tracking hundreds of millions or billions of URLs. Similarly, although Bloom-filter-based approaches introduce false positives, they provide substantial memory savings that can outweigh the loss in crawl coverage for large-scale applications.

Another important consideration is the impact of false positives on the crawling process itself. Unlike false negatives, which could lead to repeated crawling of the same resource, false positives simply prevent some

previously unseen URLs from being visited. As a result, Bloom filters do not compromise the correctness or stability of the crawler. Instead, they trade a small reduction in coverage for significant improvements in memory efficiency. The empirical results demonstrate that this trade-off is particularly favourable in the case of the Scalable Bloom Filter, which maintained a low and predictable false positive rate while requiring only a fraction of the memory consumed by a hash set.

Finally, it should be noted that the experiments were conducted using synthetic datasets to enable controlled and reproducible measurements. Although this methodology allows accurate evaluation of false positive behaviour, real-world web crawling environments may exhibit additional challenges, such as highly skewed URL distributions, URL canonicalization issues, and dynamic content generation. Future work could therefore extend the evaluation to larger-scale distributed crawling scenarios in order to further validate the observed results under production-like conditions.

7 Conclusion

This work demonstrates that Bloom filters represent a compelling solution to the duplicate URL detection problem inherent to large-scale web crawling. Through the implementation and evaluation of three variants: Standard, Counting, and Scalable. It has shown that probabilistic data structures can achieve memory reductions of over an order of magnitude relative to exact hash set approaches, while maintaining false positive rates well below practical thresholds.

7.1 When to Use Each Variant

Choosing the right filter variant depends on three primary factors: whether the dataset size is known in advance, whether deletions are required, and the memory budget available.

The **Standard Bloom filter** is the right choice when the expected number of URLs is known ahead of time and the crawl is append-only, that is, once a URL is marked visited, it will never need to be removed. This covers the majority of practical crawling scenarios: batch crawls with a fixed seed list, domain-scoped crawls, or any pipeline where the frontier size can be reasonably estimated. At roughly 10 bits per element, it offers the best memory efficiency of the three variants and should be the default unless a specific limitation rules it out.

The **Counting Bloom filter** becomes the appropriate choice when URL expiry or deletion is a system requirement. Concretely, this arises when crawling environments where pages are known to disappear (HTTP 404), domains become intermittently unreachable, or cached content has a defined time-to-live after which a URL should be eligible for recrawling. Without deletion support, a standard filter would accumulate stale entries indefinitely, requiring periodic full reconstruction is an expensive operation at scale. The fourfold memory overhead relative to the standard variant is the direct cost of this flexibility and should be weighed against how frequently deletions are actually expected. If deletions are rare or nonexistent, the standard filter is preferable.

The **Scalable Bloom filter** is warranted when the total number of URLs cannot be estimated in advance. This is common in broad, open-ended crawls that follow outbound links without a fixed domain scope, where the frontier grows unpredictably. Pre-sizing a standard or counting filter in this setting forces a difficult trade-off: underestimating n causes the false positive rate to degrade rapidly as the filter fills beyond its design capacity, while overestimating wastes memory on unused capacity. The scalable variant avoids this dilemma by growing on demand, at the cost of slightly higher query latency (all layers must be checked) and a less predictable memory footprint. It is also the most complex to reason about in terms of false positive guarantees, since the global rate depends on the tightening ratio r across layers.

It is always a good idea to start with the standard filter. Change to counting only if deletions are genuinely needed. Move to Scalable only if the dataset size is truly unbounded. In practice, however, neither variant alone fully captures the demands of a real-world web crawler. The ideal solution for this context would combine the properties of both the **Counting and Scalable** filters: supporting deletions for stale or expired URLs while also adapting dynamically to an unpredictable and potentially unbounded frontier. Such a hybrid would handle the full complexity of production crawling without requiring upfront size estimates or sacrificing the ability to remove entries, at the cost of greater implementation complexity and a less predictable memory footprint.

7.2 Summary

The Standard Bloom filter proves optimal for fixed scale crawls where memory is the primary concern, consuming approximately 610 KB for 500,000 URLs compared to over 30 MB for a standard `unordered_set`. The counting variant trades a fourfold memory increase for deletion support, enabling URL expiry and recrawling workflows that would otherwise require expensive full filter reconstructions. The scalable variant sacrifices the compactness of a pre-dimensioned structure in exchange for robustness to unknown dataset sizes, making it the most appropriate choice for open-ended crawls where the frontier cannot be bounded in advance.

Several directions remain open for future work. Extending the crawler to support HTTPS via TLS would substantially broaden its applicability. Replacing the regex-based link extractor with a proper HTML parser would improve recall on complex pages. Finally, distributing the Bloom filter across nodes using consistent hashing would be a natural next step for scaling beyond a single machine.

In summary, Bloom filters remain, over fifty years after their introduction, one of the most practically effective tools in the systems engineer's toolkit for problems where approximate membership testing at scale is acceptable, and web crawling is a canonical example of precisely such a problem.

References

[1] Wikipedia contributors. Bloom filter — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Bloom_filter, 2024. Accessed: 2026-06-03.

[2] Li Fan, Pei Cao, Jussara Almeida, and Andrei Z. Broder. Summary cache: A scalable wide-area web cache sharing protocol. *IEEE/ACM Transactions on Networking*, 8(3):281–293, June 2000. doi:10.1109/90.851975.

[3] Paulo Sérgio Almeida, Carlos Baquero, Nuno Preguiça, and David Hutchison. Scalable bloom filters. In *Proceedings of the 6th International Workshop on Information Processing in Sensor Networks (IPSN)*, pages 255–264. Springer, 2007. doi:10.1007/978-3-540-71661-7_16.

[4] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, 26(2):1–26, June 2008. doi:10.1145/1365815.1365816.

[5] Allan Heydon and Marc Najork. Mercator: A scalable, extensible web crawler. In *Proceedings of the 8th International World Wide Web Conference (WWW)*, pages 219–229. Elsevier, 1999. doi:10.1023/A:1019213109274.

[6] Sebastiano Vigna. Splitmix64 - a c++ implementation. https://github.com/svaarala/duktape/blob/master/misc/splitmix64.c, 2015. Accessed: 2026-06-03.

[7] EDA Group 1. Eda, 2026. Accessed : 2026-06-03. URL: https://github.com/oManuelmo/eda.

8 Annexes

Table 1: Standard Bloom Filter False Positive Rate (%)

URLs / Bits	500K	1M	5M
100K	1.2%	0.04%	<0.0001%
300K	29.7%	5.6%	0.0025%
500K	54.1%	21.4%	0.04%
1M	76.8%	54.2%	1.2%
2M	88.4%	76.8%	12.8%

Table 2: Scalable Bloom Filter False Positive Rate ($p = 1\%$)

Bits	**FP Rate**
100K	0.15%
300K	0.14%
500K	0.14%
1M	0.14%
2M	0.13%

Table 3: Scalable Bloom Filter False Positive Rates (%) by Target p

Bits / p	**25%**	**10%**	**5%**	**1%**	**0.1%**
100K	3.5%	1.5%	0.74%	0.15%	0.01%
300K	3.5%	1.4%	0.72%	0.14%	0.01%
500K	3.6%	1.4%	0.71%	0.14%	0.01%
1M	3.7%	1.4%	0.70%	0.14%	0.01%
2M	3.8%	1.4%	0.71%	0.13%	0.01%

Table 4: Scalable Bloom Filter Memory Usage (KB) by Target p

Bits / p	**25%**	**10%**	**5%**	**1%**	**0.1%**
100K	85	105	121	156	208
300K	256	317	364	470	624
500K	426	528	603	784	1,040
1M	853	1,057	1,207	1,569	2,081
2M	1,707	2,115	2,414	3,138	4,161

Table 5: Three-Model Benchmark (500K URLs)

Model	**FP Rate**	**Size (KB)**	**Time (ms)**
Bloom Filter (BF)	0.12%	305	466
Scalable Bloom Filter (SBF)	0.14%	784	569
Hash Set	0.0%	29,003	467